

余弦関数近似の範囲縮約モデルに関する研究開発報告

Original π -switch Formulation から `nearbyint_pi` 方式への発展

技術報告書

2026 年 5 月 31 日

概要

本報告書は、余弦関数近似の高速化を目的として考案した Original π -switch Formulation を出発点とし、その解析、課題の発見、改善、および再評価の過程を整理する。公開済み論文や他者の既存研究を再現するものではない。初期段階では Apple M3 Pro 環境で π -switch と Taylor/Horner 系近似を組み合わせ、`std::cos` に対して大きな速度差を観測した。その後、Windows 11、MSVC 19.44、x64 環境で診断測定を行い、広い入力範囲では範囲縮約の選択が総合性能を支配することを確認した。

測定対象を多項式評価、範囲縮約、総合関数、呼出し方式に分離した結果、Taylor N=7 および N=8 の多項式評価自体は約 1.6–2.0 ns/sample であり、速度低下の主要因ではないことが分かった。主要因は `std::remainder` を含む範囲縮約であり、 $[-1e6, 1e6]$ では $\pi/2$ folding 縮約単体が約 22.5 ns/sample を要した。一方、bounded-input 用途を仮定した `nearbyint(x / pi)` 方式では、縮約後の区間を概ね $[-\pi/2, \pi/2]$ に保ち、精度と速度を改善できた。Apple clang による追加測定では、 $[-10^6, 10^6]$ において Taylor N=7 と `nearbyint_pi` の組合せが `std::cos` に対し -03 で 3.78 倍、-03 -ffast-math で 4.91 倍の速度比を示した。一方、約 16 倍の速度比は fast-math 下の inline direct な polynomial-only 測定でのみ観測され、汎用入力向け高精度関数の結果ではなかった。

以上から、初期 π -switch の着想は有用であったが、高精度化には縮約区間の再設計が必要である。将来の実装では bounded-input API と general-input API を分離し、範囲縮約方式を明示的な policy として扱う必要がある。

目次

1	Motivation	2
2	Mathematical Background	3
2.1	標準関数と近似関数	3
2.2	研究開発上の分離課題	3
3	Original π -switch Model	4
3.1	Original π -switch Formulation	4
3.2	Reference: 2π reduction	4
4	Analysis of the Original Model	4
4.1	縮約区間と Taylor 剰余項	4
5	Improved Range Reduction	5
5.1	$\pi/2$ folding	5

5.2	nearbyint_pi 方式	5
6	Polynomial Approximation	5
6.1	Horner 法	5
6.2	Estrin 法	5
7	Error Analysis	5
7.1	指標	5
7.2	観測結果	6
8	Benchmark Design	6
8.1	測定環境	6
8.2	測定対象の分離	7
9	Results	7
9.1	Polynomial-only benchmark	7
9.2	呼出し方式	7
9.3	Range reduction benchmark	8
9.4	総合性能	8
9.5	macOS Apple clang 追加測定	9
10	Discussion	10
10.1	Windows/MSVC での速度低下原因	10
10.2	解析結果の分類	10
10.3	M3 Pro 結果と Windows/MSVC 結果の相違点	11
11	Future Work toward Library Design	12
11.1	API の分離	12
11.2	実装層	12
12	限界	12
13	結論	12
付録 A	生成された図表一覧	13
付録 B	レポート内で使用した全数式一覧	13
付録 C	今後追加すべき実験一覧	14

1 Motivation

本研究の目的は、Taylor 展開と余弦関数の周期性を利用し、再現可能な条件下で高速な余弦近似を設計することである。初期仮説を保護することは目的ではない。解析の結果として、何が成立し、何が限定条件付きで成立し、何が棄却され、何が未解決かを分類する。対象は `double` 精度の余弦関数に限定する。

float、固定小数点、SIMD intrinsic、Minimax 係数は将来課題とする。

検証では、次の原則を採用する。

1. 速度と精度を別々に評価する。
2. 範囲縮約と多項式評価を分離して測定する。
3. 環境依存の観測を普遍的な主張へ一般化しない。
4. 不都合な結果も除外せず、CSV に保存する。
5. ULP 誤差はゼロ近傍で大きくなり得るため、絶対誤差と併記する。

2 Mathematical Background

2.1 標準関数と近似関数

`std::cos` の内部実装は、標準ライブラリ、CPU、コンパイラ、浮動小数点フラグに依存する。したがって、近似関数との速度比較は、CPU、OS、コンパイラ、フラグ、入力分布、試行回数を固定した観測として記述する必要がある。

余弦関数の Maclaurin 展開は

$$\cos x = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k)!} x^{2k} \quad (1)$$

である。本報告書では、打ち切り多項式を

$$P_N(x) = \sum_{k=0}^N \frac{(-1)^k}{(2k)!} x^{2k} \quad (2)$$

と定義する。したがって、Taylor N=7 は x^{14} まで、Taylor N=8 は x^{16} までを含む。

Taylor の定理から、適切な ξ に対する剰余項は

$$R_N(x) = \cos x - P_N(x) = \frac{(-1)^{N+1} \cos \xi}{(2N+2)!} x^{2N+2}, \quad |R_N(x)| \leq \frac{|x|^{2N+2}}{(2N+2)!} \quad (3)$$

と評価できる。よって、多項式次数だけでなく、縮約後の最大 $|x|$ が重要である。

2.2 研究開発上の分離課題

初期提案の解析から、 π -switch、 $\pi/2$ folding、Taylor 多項式、Horner 法、Estrin 法、および M3 Pro での速度差を一括して扱わず、少なくとも次の点を分離する必要があると分かった。

- π -switch の恒等式としての正しさと、近似精度への効果
- 多項式評価そのものの速度と、範囲縮約を含む総合速度
- 特定 M3 Pro 環境の観測と、クロスプラットフォーム一般化

3 Original π -switch Model

3.1 Original π -switch Formulation

本研究の出発点は、次式である。

$$f(\theta) = (-1)^{\lfloor \theta/\pi \rfloor} \sum_{k=0}^{10} \frac{(-1)^k}{(2k)!} \left(\theta - \pi \left\lfloor \frac{\theta}{\pi} \right\rfloor \right)^{2k} \quad (4)$$

式 (4) を本報告書では Original π -switch Formulation と呼ぶ。この発想は

$$m = \left\lfloor \frac{\theta}{\pi} \right\rfloor, \quad t = \theta - \pi m, \quad \cos \theta = (-1)^m \cos t, \quad 0 \leq t < \pi \quad (5)$$

という恒等式に基づく。初期仮説は、周期性によって Taylor 評価点を縮約すれば、標準関数より少ない処理で十分な精度を得られる可能性がある、というものであった。

3.2 Reference: 2π reduction

比較用の基準モデルとして

$$r_{2\pi}(x) = \text{remainder}(x, 2\pi) \quad (6)$$

を用いる。戻り値は概ね $[-\pi, \pi]$ に入る。

4 Analysis of the Original Model

4.1 縮約区間と Taylor 剰余項

式 (5) は恒等式として正しい。しかし、Taylor 展開中心を 0 とする場合、 t は最大で π に近づく。したがって、 π -switch 単体では Taylor 剰余項を十分に抑えられない。解析の結果、初期モデルには次の課題があると整理した。

1. 縮約後の区間 $[0, \pi)$ は Taylor 展開中心 0 から遠い点を含む。
2. 偶奇判定方法によっては範囲縮約のコストが増える。
3. 恒等式の正しさは、有限次数 Taylor 近似の精度を保証しない。

旧実装では偶奇判定に `std::fmod` を使用していた。診断版では整数化できる範囲において整数剰余へ置換した。Listing 4.1 に改善版の要点を示す。

Listing 1 整数偶奇判定を用いる π -switch 縮約

```
1 inline ReducedAngle reduce_pi_switch_integer(double x) {
2     const double q = std::floor(x / kPi);
3     const double reduced = x - q * kPi;
4     const auto integer_q = static_cast<std::int64_t>(q);
5     return {reduced, integer_q % 2 != 0 ? -1.0 : 1.0};
6 }
```

5 Improved Range Reduction

5.1 $\pi/2$ folding

Taylor 剰余項を抑えるため、対称性により

$$u(x) \in [-\pi/2, \pi/2], \quad \cos x = s(x) \cos u(x), \quad s(x) \in \{-1, 1\} \quad (7)$$

となるように折り返す。従来版は `std::remainder(x, 2*pi)` の後に条件分岐を適用していた。精度は良好だが、Windows/MSVC では広い入力範囲で `std::remainder` のコストが支配的になった。

5.2 `nearbyint_pi` 方式

bounded-input 用途の候補として

$$q = \text{nearbyint}\left(\frac{x}{\pi}\right), \quad v = x - q\pi, \quad \cos x = (-1)^q \cos v \quad (8)$$

を測定した。通常の丸めモードでは v は概ね $[-\pi/2, \pi/2]$ に入る。この方式は高速だが、巨大入力に対する production-grade 縮約ではない。適用可能な入力上限を API 契約として定める必要がある。

6 Polynomial Approximation

6.1 Horner 法

$y = x^2$ とおくと、式 (2) は

$$P_N(x) = c_0 + y(c_1 + y(c_2 + \cdots + yc_N) \cdots) \quad (9)$$

と評価できる。Horner 法は乗算回数が少ないが、依存チェーンが長い。

6.2 Estrin 法

Estrin 法では多項式を木構造に分割する。例えば $N=7$ では

$$\begin{aligned} P_7(x) = & (c_0 + c_1y) + (c_2 + c_3y)y^2 \\ & + ((c_4 + c_5y) + (c_6 + c_7y)y^2)y^4 \end{aligned} \quad (10)$$

と評価できる。命令レベル並列性を増やせる可能性がある一方、CPU、コンパイラ、次数により有利不利は変わる。

7 Error Analysis

7.1 指標

入力集合 $\{x_i\}_{i=1}^M$ 、参照値 $c_i = \cos x_i$ 、近似値 $\hat{c}_i$ に対し、絶対誤差を

$$e_i = |\hat{c}_i - c_i| \quad (11)$$

と定義する。集計値は

$$E_{\max} = \max_i e_i, \quad E_{\text{mean}} = \frac{1}{M} \sum_{i=1}^M e_i \quad (12)$$

および

$$E_{\text{RMSE}} = \sqrt{\frac{1}{M} \sum_{i=1}^M (\hat{c}_i - c_i)^2} \quad (13)$$

とする。相対誤差は

$$E_{\text{rel},i} = \frac{|\hat{c}_i - c_i|}{|c_i|} \quad (14)$$

であるが、 $\cos x \approx 0$ で不安定になるため、主要指標にはしない。

ULP 距離は浮動小数点表現を符号順序に対応する整数へ写像した $I(\cdot)$ を用いて

$$E_{\text{ULP},i} = |I(\hat{c}_i) - I(c_i)| \quad (15)$$

と定義する。これもゼロ近傍では大きくなり得るため、絶対誤差と併記する。

7.2 観測結果

表 1 に主要な最大絶対誤差を示す。Taylor N=7 および N=8 は $\pi/2$ 近傍まで縮約すれば高精度である。一方、 π -switch 単体の Taylor N=7 は約 4.17×10^{-6} であり、高精度版として扱うべきではない。

表 1 主要モデルの最大絶対誤差

入力範囲	モデル	最大絶対誤差
$[-\pi/2, \pi/2]$	Taylor N=7 + $\pi/2$ folding	6.51×10^{-11}
$[-\pi/2, \pi/2]$	Taylor N=8 + $\pi/2$ folding	5.26×10^{-13}
$[-10^6, 10^6]$	Taylor N=7 + $\pi/2$ folding	1.04×10^{-10}
$[-10^6, 10^6]$	Taylor N=8 + $\pi/2$ folding	3.95×10^{-11}
$[-10^6, 10^6]$	Taylor N=7 + π -switch integer	4.17×10^{-6}
$[-10^6, 10^6]$	Taylor N=7 + <code>nearbyint_pi</code>	1.60×10^{-10}
$[-10^6, 10^6]$	Taylor N=8 + <code>nearbyint_pi</code>	9.75×10^{-11}

初期実装の掲載コード相当の最大絶対誤差は約 1.83×10^{-3} であった。また、初期ノート本文は「Taylor N=10、第 20 乗まで」と説明していたが、掲載コードの最高項は $x^{10}/10!$ であった。本文とコードは一致していない。この差異は研究過程の記録として残し、以後は式 (2) の N と最高次数 $2N$ を明示する。

8 Benchmark Design

8.1 測定環境

Windows/MSVC 確認測定の条件を表 2 に示す。

この環境では clang-cl および MinGW/GCC は検出されなかった。したがって、コンパイラ横断比較は未完了である。

表 2 Windows/MSVC 確認測定環境

項目	値
OS	Windows 11, 10.0.22631
CPU	Intel64 Family 6 Model 183 Stepping 1
compiler	MSVC 19.44.35211, x64
flags	/O2 /fp:precise, /O2 /fp:fast
入力生成	各範囲の一様乱数、固定 seed
確認測定	2,000,000 samples、warm-up 3 回、trial 9 回
代表値	trial 中央値
計時	<code>std::chrono::steady_clock</code>
最適化抑止	集計値を <code>volatile sink</code> へ格納

8.2 測定対象の分離

診断ベンチでは次を別々に測定した。

1. `polynomial_only`: 既に $[-\pi/2, \pi/2]$ へ縮約済みの入力
2. `reduction_only`: 範囲縮約のみ
3. `full`: 範囲縮約と多項式評価の合計
4. `call_style`: 関数ポインタ、直接呼出し、`template`、`lambda`

9 Results

9.1 Polynomial-only benchmark

表 3 に、`/O2 /fp:fast` での多項式単体の確認測定を示す。

表 3 縮約済み入力に対する多項式単体性能

モデル	ns/sample	<code>std::cos</code> 比	最大絶対誤差
Taylor N=7 Horner	1.565	3.90x	6.51×10^{-11}
Taylor N=7 Estrin	1.614	3.78x	6.51×10^{-11}
Taylor N=8 Horner	1.913	3.19x	5.26×10^{-13}
Taylor N=8 Estrin	1.823	3.35x	5.26×10^{-13}

多項式単体は約 1.6-2.0 ns/sample である。したがって、広い入力範囲における大幅な速度低下の主要因ではない。Horner 法と Estrin 法の順位は次数とフラグで入れ替わり、一方が常に優れるとは言えない。

9.2 呼出し方式

表 4 に Taylor N=7 Horner の呼出し方式比較を示す。

表 4 呼出し方式比較: Taylor N=7 Horner、/O2 /fp:fast

呼出し方式	ns/sample
function pointer	1.565
inline direct	1.559
template functor	1.569
lambda inline	1.631

関数ポインタは将来のインライン化やベクトル化を阻害し得る。ただし、今回の多項式単体測定では差は小さく、広い入力範囲での大幅な速度低下の主要因ではなかった。

9.3 Range reduction benchmark

表 5 に /O2 /fp:fast で の縮約単体性能を示す。

表 5 範囲縮約単体の性能: /O2 /fp:fast、単位 ns/sample

モデル	$[-\pi, \pi]$	$[-100, 100]$	$[-10^6, 10^6]$
<code>std::remainder(x, 2*pi)</code>	2.305	17.020	18.119
<code>std::fmod(x, 2*pi)</code>	2.068	3.575	3.278
<code>floor(x / pi)</code>	1.239	1.218	1.207
<code>nearbyint(x / pi)</code>	1.973	1.961	1.865
π -switch + fmod parity	9.727	14.525	13.848
π -switch + integer parity	9.307	9.024	8.948
$\pi/2$ folding + remainder	6.929	21.845	22.470
<code>nearbyint_pi</code> + integer parity	7.103	7.969	7.389

`std::remainder` は $[-100, 100]$ 以上で急激に高コストになった。また、 π -switch の偶奇判定を `std::fmod` から整数剰余へ変えると、広い範囲で縮約コストが改善した。ただし、 π -switch 自体の精度不足は残る。

9.4 総合性能

表 6 および図 1 に、/O2 /fp:fast で の主要総合性能を示す。

/O2 /fp:precise では、 $[-10^6, 10^6]$ における $N=7 + \text{nearbyint_pi}$ は 10.084 ns/sample、`std::cos` は 9.280 ns/sample であり、速度比は 0.92x だった。bounded-input 候補の小幅な優位性も、環境とフラグに依存する観測として扱う必要がある。

表 6 主要モデルの総合性能: /O2 /fp:fast

入力範囲	モデル	ns/sample	std::cos 比	最大絶対誤差
$[-\pi/2, \pi/2]$	std::cos	6.103	1.00x	0
$[-\pi/2, \pi/2]$	N=7 + nearbyint_pi	3.288	1.86x	6.51×10^{-11}
$[-\pi, \pi]$	std::cos	8.665	1.00x	0
$[-\pi, \pi]$	N=7 + nearbyint_pi	7.778	1.11x	6.51×10^{-11}
$[-100, 100]$	std::cos	8.467	1.00x	0
$[-100, 100]$	N=7 + $\pi/2$ folding	23.299	0.36x	6.51×10^{-11}
$[-100, 100]$	N=7 + nearbyint_pi	8.423	1.01x	6.51×10^{-11}
$[-10^6, 10^6]$	std::cos	9.539	1.00x	0
$[-10^6, 10^6]$	N=7 + $\pi/2$ folding	22.184	0.43x	1.04×10^{-10}
$[-10^6, 10^6]$	N=7 + π -switch integer	9.479	1.01x	4.17×10^{-6}
$[-10^6, 10^6]$	N=7 + nearbyint_pi	8.801	1.08x	1.60×10^{-10}
$[-10^6, 10^6]$	N=8 + nearbyint_pi	8.886	1.07x	9.75×10^{-11}

モデル	相対的な棒長	ns/sample
[-10 ⁶ , 10 ⁶]、/O2 /fp:fast		
std::cos		9.539
N7 + half-pi remainder		22.184
N7 + pi-switch integer		9.479
N7 + nearbyint-pi		8.801
N8 + nearbyint-pi		8.886

図 1 広い入力範囲における総合コスト。短い棒ほど高速である。

9.5 macOS Apple clang 追加測定

MacBook Pro M3 Pro 上で、Apple clang を用いて同一診断ベンチを実行した。各条件は 2,000,000 samples、warm-up 3 回、trial 9 回、trial 中央値である。表 7 に広い入力範囲の主要結果を示す。

π -switch 版は高速であるが、N=7 の最大絶対誤差は約 4.17×10^{-6} である。高精度版として採用する根拠にはならない。一方、nearbyint_pi 版は $[-10^6, 10^6]$ で N=7 が約 1.04×10^{-10} 、N=8 が約 3.95×10^{-11} の最大絶対誤差を示した。

また、fast-math 下の call-style 測定では、縮約済み入力に対する N=7 Horner の inline direct 呼出しが std::cos に対して 16.84 倍となった。これは polynomial-only の観測であり、range reduction を含む汎用入力向け高精度関数の速度比ではない。

表 7 macOS Apple clang における総合性能: $[-10^6, 10^6]$

モデル	-O3		-O3 -ffast-math	
	ns/sample	std::cos 比	ns/sample	std::cos 比
std::cos	9.485	1.00x	9.786	1.00x
N=7 + $\pi/2$ folding	75.654	0.13x	75.800	0.13x
N=7 + π -switch fmod	1.793	5.29x	1.871	5.23x
N=7 + π -switch integer	2.395	3.96x	2.128	4.60x
N=7 + nearbyint_pi	2.510	3.78x	1.993	4.91x
N=8 + nearbyint_pi	2.772	3.42x	2.110	4.64x

10 Discussion

10.1 Windows/MSVC での速度低下原因

診断結果から、原因候補を表 8 のように分類する。

表 8 Windows/MSVC における速度低下原因の分類

候補	判定	根拠
Taylor 多項式自体	主因ではない	単体で約 1.6–2.0 ns/sample。
std::remainder	主要因	広い範囲で縮約単体が約 18 ns/sample、folding 込みで約 22.5 ns/sample。
π -switch の fmod 偶奇判定	副次的要因	整数剰余へ変えると広い範囲で明確に改善。
関数ポインタ	今回の主因ではない	多項式単体では直接呼出しとの差が小さい。ただし将来の最適化を阻害し得る。
MSVC の std::cos	比較的強い	広い範囲でも約 8.5–9.5 ns/sample。標準ライブラリ依存の観測。
SIMD/FMA	未活用	MSVC 生成コードで vfmadd は 0 件。対象ループの自動ベクトル化も未確認。
Apple clang の最適化	要追加解析	Mac では floor、fmod、nearbyint 系縮約が低コスト。生成コード確認が必要。

10.2 解析結果の分類

10.2.1 生き残った仮説

- π -switch の式 (5) は恒等式として正しい。
- Taylor N=7/N=8 の多項式評価自体は高速である。
- $\pi/2$ 近傍まで縮約すると Taylor 近似の誤差は小さくなる。

- Horner 法と Estrin 法の優劣は CPU、コンパイラ、次数に依存する。

10.2.2 限定条件付きとなった仮説

- M3 Pro で大きな速度差が観測されたという結果は、polynomial-only か full pipeline かを区別した上で保持する必要がある。
- bounded-input 条件では `std::cos` に勝てる可能性があるが、Windows/MSVC で観測した差は小さい。

10.2.3 解析により棄却された初期仮説

- π -switch 単体で高精度化できる。
- π -switch により理論上約 10^6 倍の精度向上が保証される。
- 初期掲載コードが Taylor $N=10$ 、 x^{20} までを評価する。
- Horner 法が常に Estrin 法より高速である。
- 現行 Windows/MSVC 実装が SIMD/FMA を最大限活用している。

10.2.4 追加検証が必要な仮説

- Apple clang で range reduction が低コストとなった直接原因
- Apple clang、clang-cl、GCC、MinGW/GCC での比較
- SIMD backend、明示的 FMA、Minimax 係数の効果
- `nearbyint_pi` 方式を許容できる最大入力範囲
- sin 専用近似および sin/cos 同時計算 API の性能

10.3 M3 Pro 結果と Windows/MSVC 結果の相違点

M3 Pro で追加測定した CSV により、Apple clang では `nearbyint_pi` 方式が Windows/MSVC より明確に低コストであることを確認した。一方、初期ノートに記録された最大約 16 倍という速度差については、今回の full pipeline 測定では再現しなかった。

Windows/MSVC では、従来の $\pi/2$ folding + `std::remainder` 版は、広い入力範囲で `std::cos` より遅かった。この差は少なくとも Windows/MSVC では範囲縮約コストにより説明できる。M3 Pro では、 $N=7$ + `nearbyint_pi` の full pipeline は `-O3` で 3.78 倍、`-O3 -ffast-math` で 4.91 倍だった。一方、polynomial-only の inline direct 呼出しでは 16.84 倍を観測した。したがって、初期観測は縮約済み入力または縮約コストが小さい条件の性能を強く反映した可能性がある。直接原因の確定には Apple clang 生成コードの解析が必要である。

したがって、現時点で妥当な表現は次の通りである。

M3 Pro 上の Apple clang、`-O3 -ffast-math`、縮約済み入力、inline direct 呼出しという条件では、Taylor $N=7$ Horner が `std::cos` に対して 16.84 倍の速度比を示した。range reduction を含む $N=7$ + `nearbyint_pi` の full pipeline では、同じ入力範囲 $[-10^6, 10^6]$ で 4.91 倍だった。

11 Future Work toward Library Design

11.1 API の分離

ライブラリ化前に、少なくとも bounded-input と general-input を分離する。概念的な API を Listing 11.1 に示す。

Listing 2 将来 API の概念案

```
1 enum class precision_policy { fast, balanced, accurate };
2 enum class reduction_policy { bounded_input, general_input };
3
4 double cos_approx(double x,
5                   precision_policy precision,
6                   reduction_policy reduction);
```

11.2 実装層

1. range reduction
2. polynomial coefficient set
3. Horner または Estrin evaluation
4. scalar または SIMD backend
5. public API と policy selection

bounded-input では `nearbyint_pi` 方式を候補とする。general-input では `std::remainder` 依存を外し、Cody–Waite 方式と Payne–Hanek 方式などの段階的縮約を比較する必要がある。

12 限界

- Windows 11、Intel x64、MSVC 19.44、および MacBook Pro M3 Pro、Apple clang の 2 環境で確認測定した。
- Linux、GCC、clang-cl、MinGW/GCC は未測定である。
- `double` 余弦関数のみを対象とした。
- `nearbyint_pi` は巨大入力向けの厳密縮約ではない。
- Minimax 係数、SIMD intrinsic、明示的 FMA は未評価である。
- 相対誤差はゼロ近傍で不安定なため、主要表から除外した。
- Apple clang の生成コードは未解析であり、Mac で range reduction が低コストとなった直接原因は未確定である。

13 結論

Original π -switch Formulation は、周期性を利用して多項式入力を縮約するという本研究の出発点である。解析から、恒等式としては正しい一方、縮約区間 $[0, \pi)$ が Taylor 展開中心 0 から遠い点を含むた

め、高精度化を単独では保証しないことが分かった。

診断測定から、Taylor $N=7/N=8$ 多項式そのものは高速であり、Windows/MSVC における速度低下の主要因は範囲縮約であると判断できる。特に、広い入力範囲に対する `std::remainder` のコストが支配的だった。

π -switch は恒等式として正しいが、縮約後の区間が $[0, \pi)$ であるため、Taylor 近似の高精度化を単独では保証しない。整数偶奇判定により速度は改善するが、精度問題は残る。

改善後の bounded-input 向け `nearbyint_pi` 方式は、Windows/MSVC の一部条件で `std::cos` と同等またはわずかに高速だった。MacBook Pro M3 Pro、Apple clang では、 $[-10^6, 10^6]$ において -03 で 3.78 倍、-03 `-ffast-math` で 4.91 倍の速度比を観測した。ただし、これは汎用入力への保証ではない。将来のライブラリでは bounded-input と general-input を分離し、範囲縮約を独立 policy として設計すべきである。

付録 A 生成された図表一覧

図

1. 図 1: 広い入力範囲における総合コスト

表

1. 表 1: 主要モデルの最大絶対誤差
2. 表 2: Windows/MSVC 確認測定環境
3. 表 3: 縮約済み入力に対する多項式単体性能
4. 表 4: 呼出し方式比較
5. 表 5: 範囲縮約単体の性能
6. 表 6: 主要モデルの総合性能
7. 表 7: macOS Apple clang における総合性能
8. 表 8: Windows/MSVC における速度低下原因の分類

付録 B レポート内で使用した全数式一覧

1. 式 (1): 余弦関数の Maclaurin 展開
2. 式 (2): Taylor 打ち切り多項式
3. 式 (3): Taylor 剰余項と上界
4. 式 (4): Original π -switch Formulation
5. 式 (5): π -switch の基礎恒等式
6. 式 (6): 2π reduction
7. 式 (7): $\pi/2$ folding
8. 式 (8): `nearbyint_pi` 方式
9. 式 (9): Horner 法
10. 式 (10): Estrin 法

11. 式 (11): 絶対誤差
12. 式 (12): 最大絶対誤差と平均絶対誤差
13. 式 (13): RMSE
14. 式 (14): 相対誤差
15. 式 (15): ULP 距離

付録 C 今後追加すべき実験一覧

1. Apple clang version、完全な flags、CPU 詳細を測定 CSV とともに保存する。
2. Apple clang の生成コードを保存し、range reduction のライブラリ呼出し、SIMD 命令、FMA 命令の有無を確認する。
3. Windows で clang-cl、可能なら MinGW/GCC を追加し、MSVC と比較する。
4. Linux で GCC および Clang、macOS で Apple clang を測定する。
5. /fp:precise、/fp:fast、-ffast-math の影響を比較する。
6. bounded-input の上限を段階的に拡大し、nearbyint_pi の誤差限界を測る。
7. Cody–Waite および Payne–Hanek 縮約を実装し、general-input 性能を比較する。
8. scalar、auto-vectorized loop、明示的 SIMD backend を分離して測定する。
9. Horner、Estrin、Minimax を同一区間、同一次数、同一誤差基準で比較する。
10. float、double、fixed-point を別スイートとして測定する。
11. NaN、正負 infinity、正負 zero、folding 境界直前直後をテストする。
12. sin 専用近似、位相シフト版、sin/cos 同時計算 API を比較する。